

Champion Document – TL2

Procedural Level & Obstacle Generation – Caleb Blondell

1. Introduction

TL2 owns Procedural Level & Obstacle Generation. It builds the endless, forward-only play area that the player and enemies inhabit, and it drives progression through the level. Chunks are generated ahead of the player from a deterministic seed, then recycled through a pool once the player leaves them behind. Chunk generation is abstracted behind a common interface, and obstacles, platforms, and spawn locations are placed by separate systems, so new chunk types and content can be added without changing unrelated code. TL2 also owns difficulty, which scales with distance and is encapsulated within the feature. Other features can only read it, using it to vary enemy stats, loot, and other mechanics. Public outputs are distance, difficulty, and an event raised when each chunk is spawned, carrying the information other systems need, such as spawn locations and chunk difficulty.

2. Use Case – Diagrams and Scenarios

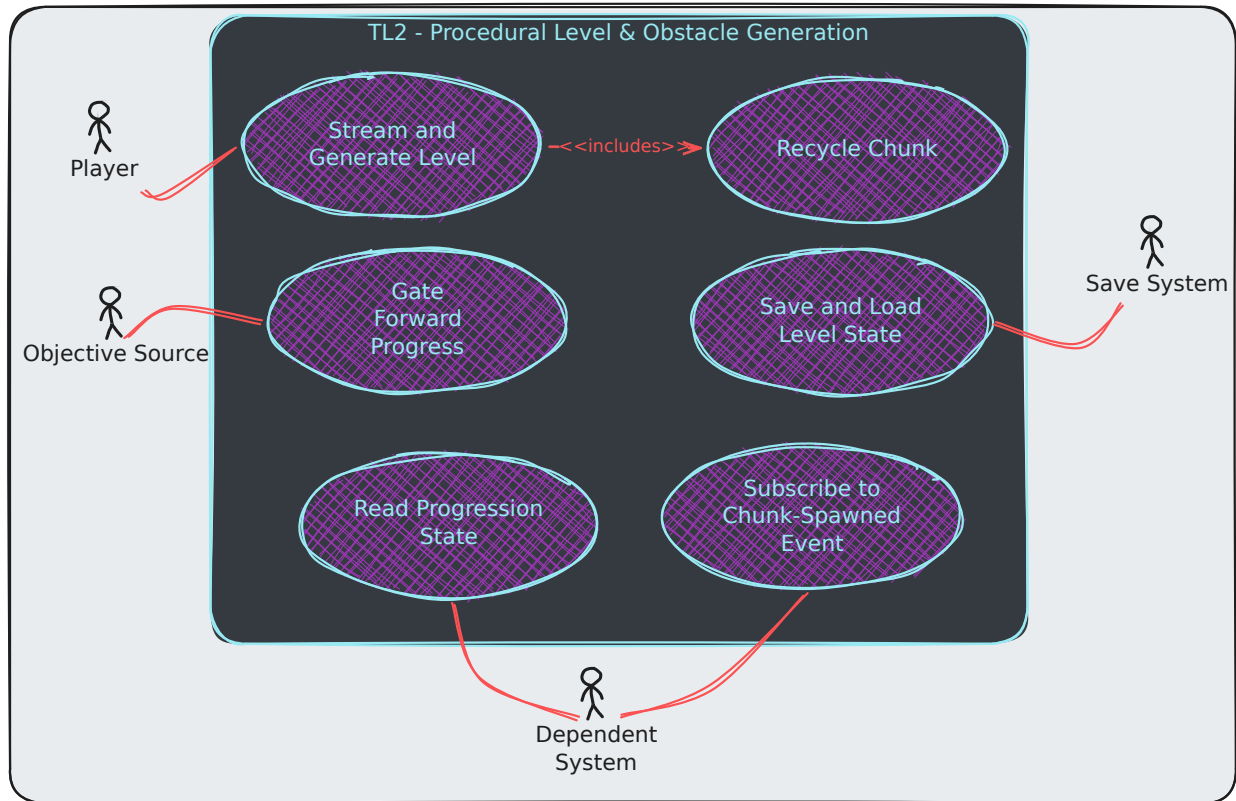
Actors

Actor	Relationship to TL2
Player (via TL6)	Source of world position that drives streaming. TL2 only reads this; it does not command the player.
Dependent system (e.g. TL4)	Reads distance and difficulty, and subscribes to the chunk-spawned event.
Objective source	Reports chunk-completion status that TL2's gate depends on.
Save system	Local persistence that TL2 reads from and writes to across sessions.

Use cases

ID	Use case	Description
UC1	Stream and generate level	As the player advances, request and generate the next chunk ahead of them from the seed and current difficulty.
UC2	Recycle chunk	Return a chunk that has fallen behind the player to the pool for reuse, without destroying or reinstantiating it.
UC3	Read progression state	Expose current distance and current difficulty as read-only values to dependent systems.
UC4	Subscribe to chunk-spawned event	Notify subscribers when a chunk is spawned, with an immutable payload of chunk data.
UC5	Gate forward progress	Withhold the forward exit of a chunk until its main level requirements are reported complete. Extends UC1.
UC6	Save and load level state	Persist and restore the minimal state (seed, furthest chunk index, difficulty state) needed to resume a run.

ID	Use case	Description
UC7	Start new run	Fallback when saved data fails validation on load: discard it and begin a new run instead of resuming. Extends UC6.



Scenario – Stream and Generate Level

Field	Detail
ID	SG1
Summary	As the player advances, TL2 requests and generates the next chunk ahead of them, and recycles chunks that fall behind.
Actors	Player (via TL6)
Preconditions	A level session is active, with an established seed and a current difficulty value.

Basic sequence

1. Player advances toward the leading edge of the active chunk window.
2. LevelStreamer detects the player is within lookahead distance of the edge.
3. LevelStreamer requests the next chunk from ChunkGenerator.

4. ChunkGenerator produces the chunk deterministically from the seed, chunk index, and current difficulty.
5. The chunk is placed into the level.
6. OnChunkSpawned fires with the chunk's bounds, difficulty, and spawn locations.
7. As the player passes the trailing edge, the oldest active chunk is deactivated and returned to ChunkPool for reuse.
8. Player continues advancing; sequence repeats from Step 1.

Exceptions

- **Step 1a – the <<extend>> case “Gate Forward Progress”:** if the current chunk's main level requirements are not yet complete when the player reaches its forward exit, the exit stays closed and the player cannot advance past it. Resumes at Step 1 once requirements are reported complete.
- **Step 4a:** the requested chunk index already exists in ChunkPool. ChunkPool supplies the pooled chunk instead of generating a new one. Resumes at Step 5.

Postconditions: The active chunk window is current; distance has increased; recycled chunks are available in the pool.

Priority: 1

Scenario – Load Level State with Invalid Save Data

Field	Detail
ID	LD1
Summary	On load, TL2 validates a save blob before applying it, and falls back safely if the data is untrusted or corrupt.
Actors	Save system
Preconditions	A save blob is present and offered to TL2 on load.

Basic sequence

1. Save system supplies a save blob to TL2 on load.
2. TL2 reads the schema version field from the blob.
3. TL2 validates the blob's schema and version against the current format.
4. TL2 reconstructs the seed, furthest chunk index, and difficulty state from the validated blob.
5. TL2 resumes the run from the reconstructed state.

Exceptions

- **Step 3a – the <<extend>> case “Start New Run”:** schema or version validation fails (missing fields, out-of-range values, unknown version). TL2 discards the blob, reports the failure rather than applying untrusted data, and starts a new run instead of resuming. Scenario ends here.

Postconditions: The run resumes from valid state, or a new run begins; no crash occurs; a validation failure is reported.

Priority: 1

Scenario – Sustained High-Speed Streaming

Field	Detail
ID	HS1
Summary	Streaming and pooling keep pace with the player at elevated speed, without visible seams, repeats, or allocation spikes.
Actors	Player (via TL6)
Preconditions	Player speed exceeds the baseline threshold; the chunk pool is populated from prior play.

Basic sequence

1. Player speed increases beyond baseline.
2. LevelStreamer recalculates lookahead distance as a function of current speed.
3. LevelStreamer requests chunks earlier, in proportion to the increased lookahead.
4. ChunkGenerator and ChunkPool supply chunks fast enough that the player never reaches an ungenerated edge.
5. Chunks recycle through ChunkPool at a correspondingly higher rate as the player passes their trailing edges.
6. No visible seam or repeat occurs across consecutive chunks despite the higher rate.

Exceptions

- **Step 3a:** the calculated lookahead would exceed the pool's available generation budget. LevelStreamer caps lookahead at a safe maximum and logs the condition. Resumes at Step 3.
- **Step 4a – the <<extend>> case “Gate Forward Progress”:** the player reaches a gated chunk boundary while at high speed. Forward progress halts until requirements are met, exactly as in normal play. Resumes at Step 4 once cleared.

Postconditions: Frame time is sustained; no seams or repeats occur; distance and difficulty continue updating correctly at high speed.

Priority: 1

3. Interface Contract

All externally visible members are typed and have observable pass/fail behavior. External features may read state or subscribe to published events; they do not directly mutate TL2 internal state, with the exception noted below.

Inputs consumed by this feature

Input	Source / notes
Player world position	Read-only, from TL6. Polled by LevelStreamer to decide when to request the next chunk.
Save data blob	From the save system, on load. Treated as untrusted; schema and version are validated before use.

Input	Source / notes
Chunk completion / objective status	Reported by whatever system owns chunk objectives, to satisfy ChunkGate.

Outputs and public interface this feature exposes

Member	Type	Notes
GetCurrentDistance()	Method → float	Read-only. Furthest forward progress reached; never decreases.
GetCurrentDifficulty()	Method → float	Read-only. Reflects TL2's internal DifficultyProgression; not settable by other features.
OnChunkSpawned	Event	Raised when a chunk is spawned. Payload is an immutable snapshot: chunk index, bounds, chunk difficulty, spawn locations.
SetDifficulty(float)	Method	Part of the RFP's frozen interface list. Reserved for non-gameplay callers (tests, run initialization) rather than exposed to arbitrary dependents, since TL2 otherwise derives and owns difficulty internally.
CreateMemento() / SetMemento(aMemento)	Methods	TL2's state-owning class acts as Originator, preserving encapsulation of save/restore.

State classes and dynamic binding

Chunk behavior is polymorphic. An abstract Chunk base class defines the shared contract (bounds, connection points, Populate(), etc.); GroundChunk, GapChunk, and RampChunk are concrete subclasses that override this behavior. Callers such as ChunkPool and LevelStreamer hold references typed as Chunk and invoke these members without knowing the concrete type – the correct override is resolved at runtime (dynamic binding). New chunk types can be added as additional subclasses without changing LevelStreamer, ChunkPool, or other chunk types, satisfying the Protected Variations requirement.

ChunkGenerator follows the same pattern as a strategy: it is called with a chunk index, seed, and difficulty, and returns a Chunk without the caller knowing which concrete generation logic produced it.

Process Description — Context Diagram

The project-wide context diagram below (team) frames TL2 as one contributor to a single game process. Player input, save data, and engine timing all flow through this shared boundary before reaching TL2 specifically.

Context Diagram: Escape the 90s

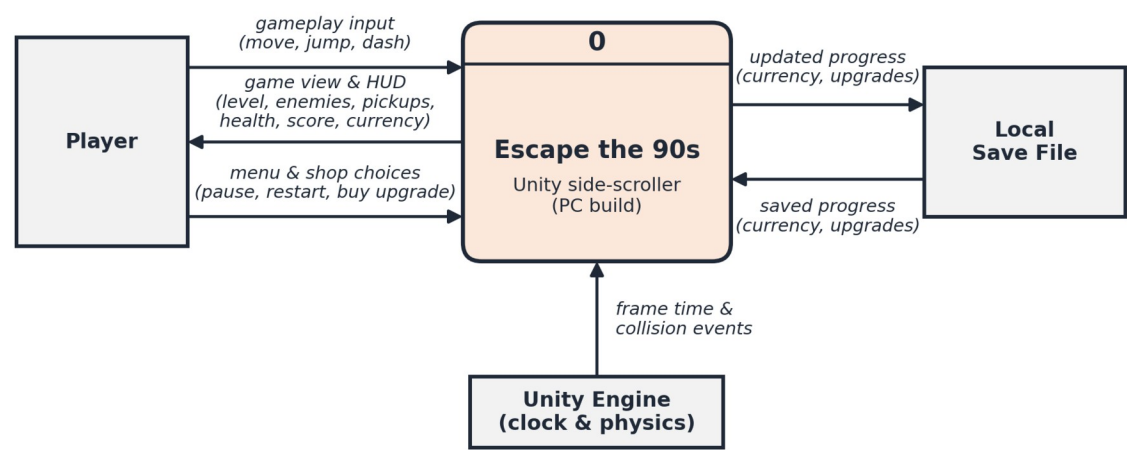


Diagram 0 decomposes Process 0 into the six features: TL1 UI/HUD & Game State, TL2 Level Generation, TL3 Power-Ups, TL4 Enemies & Combat, TL5 Meta Progression, TL6 Player Controller. Every flow above must reappear there.

Figure 1. Context diagram – Escape the 90s (team).

The diagram below narrows that same boundary to TL2 alone: process 0 is TL2 itself, with the same four external entities scoped to the flows TL2 specifically reads and writes.

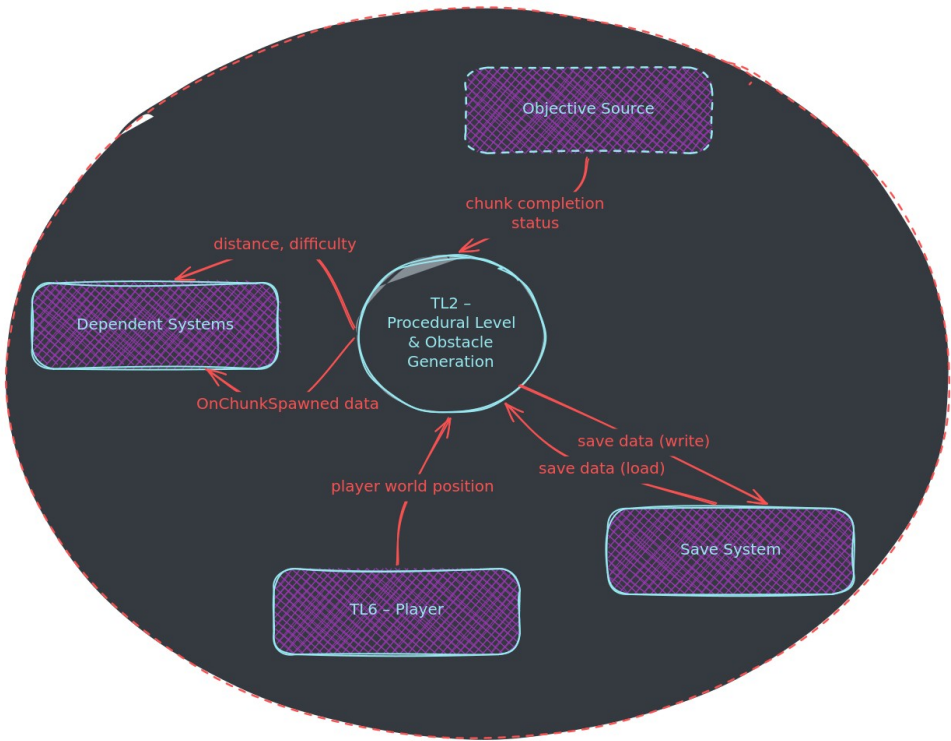


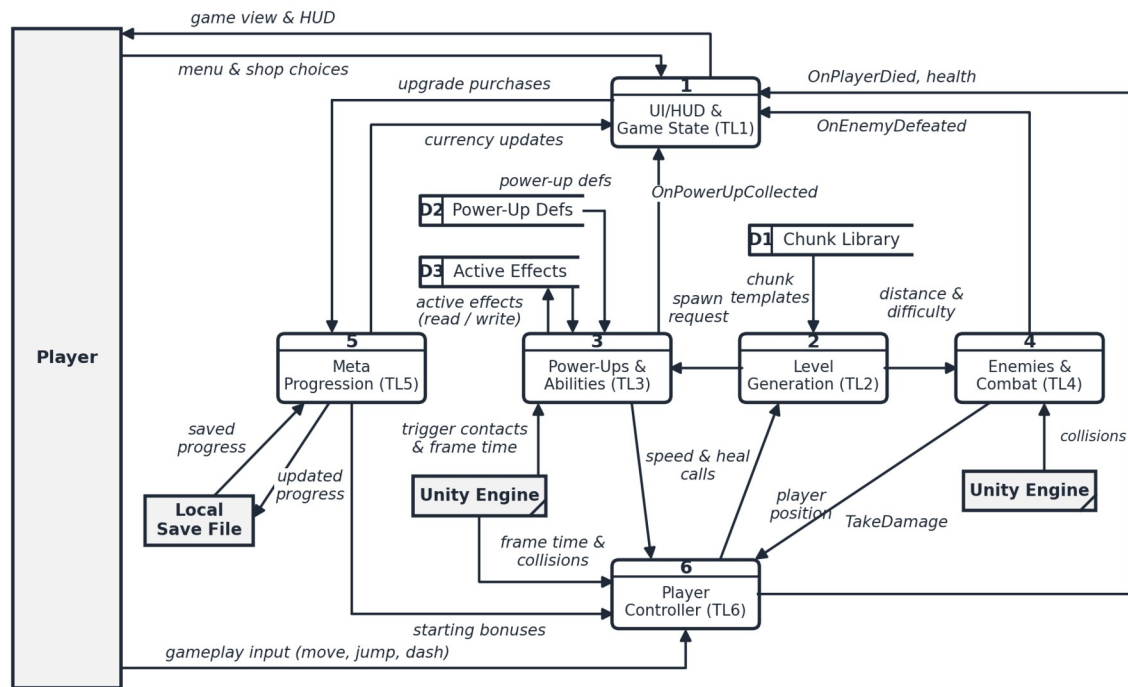
Figure 2. Context diagram, scoped to TL2 (author).

4. Data Flow Diagrams – Gane-Sarson

Diagram 0 – decomposition into the six features (team)

Diagram 0 expands the single context process into the six project features (TL1–TL6); every flow from the context diagram reappears here. TL2 (Level Generation) is one of the six process bubbles.

Diagram 0: Escape the 90s



Unity Engine appears twice (duplicate entity, marked by the corner stroke). Process numbers match TL numbers.

Figure 3. Diagram 0 – decomposition into TL1–TL6 (team).

Diagram 1 – expansion of TL2 only (author)

Diagram 1 expands only the TL2 bubble from Diagram 0, decomposing it into its own internal processes, data stores, and flows.

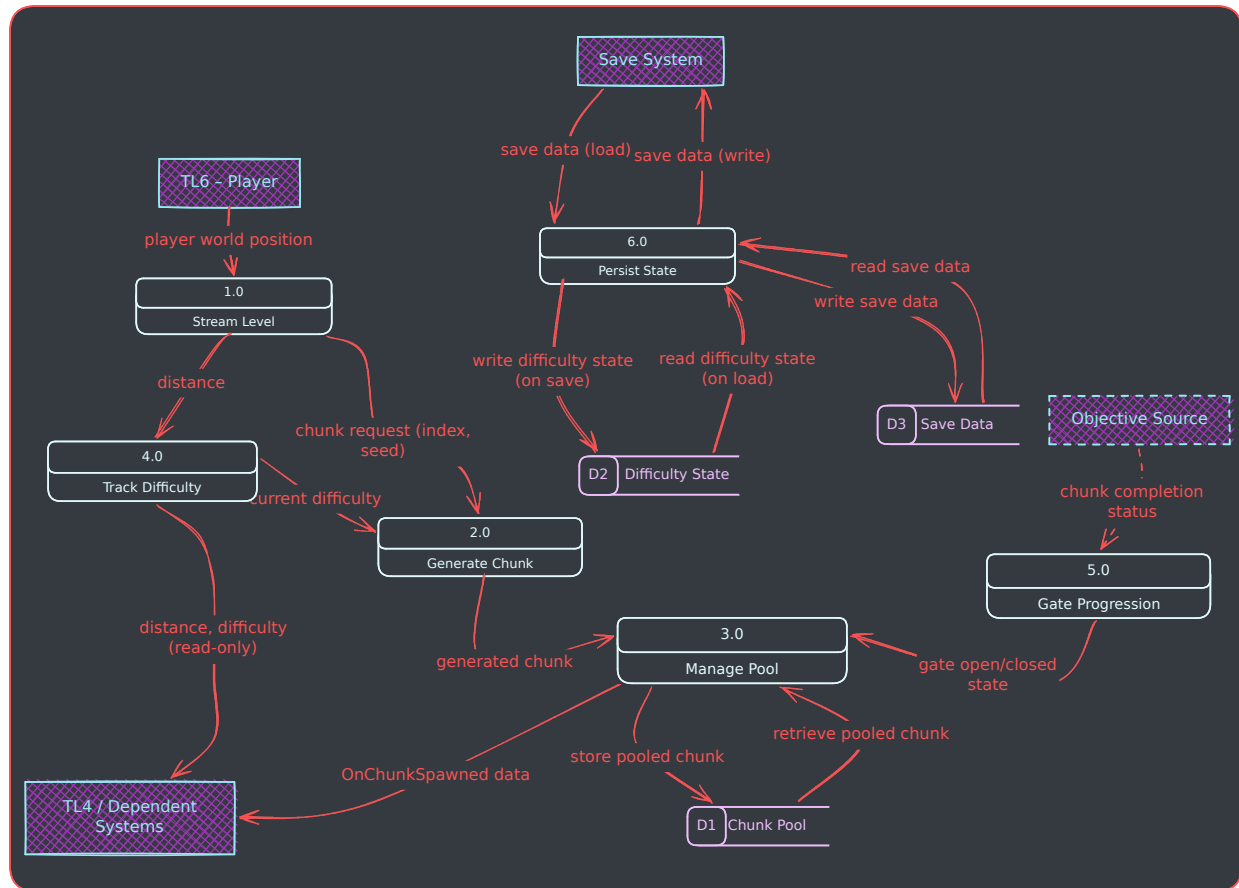


Figure 4. Diagram 1 – Level-1 DFD expanding TL2 (author).

5. Timeline

The supplied TL2 effort is 52 hours, structured as one work item per week: 6 hours in each of Weeks 1–8, and 4 hours in Week 9. Each week's hours are split into Planning, Build, and Review. Build covers whatever is actually produced that week – code for most weeks, diagrams and documentation in Week 9 – rather than being labeled “Coding,” since not every week's output is code. Review covers debugging in code-producing weeks and proofreading/consistency-checking in Week 9. Planning takes the majority share in every week, reflecting that most of the value in a short weekly session comes from deciding the right approach before producing anything. This split is a starting default and can be rebalanced per week if a particular week turns out to be more build-heavy than planning-heavy.

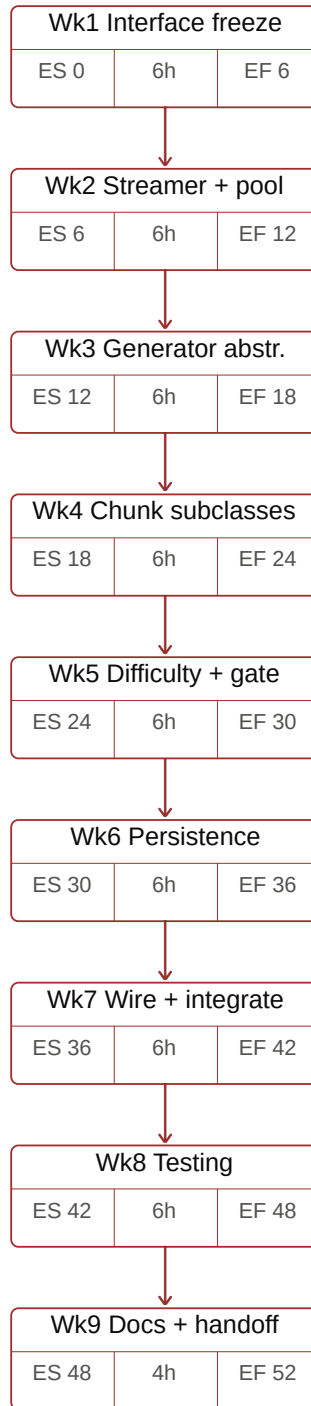
Wk	Work item	Plan	Build	Review	Total	RFP milestone
1	Interface freeze prep: confirm public contract, repo/build setup	3	2	1	6	1 – Initiation / interface freeze
2	LevelStreamer + ChunkPool skeleton; player-position read; lookahead logic	3	2	1	6	2 – Core systems

W k	Work item	Plan	Build	Review	Total	RFP milestone
3	ChunkGenerator abstraction + deterministic seeding	3	2	1	6	2 – Core systems
4	Chunk subclasses: GroundChunk, GapChunk, RampChunk + connection/seam logic	3	2	1	6	3 – Feature completion
5	DifficultyProgression (distance-driven) + ChunkGate	3	2	1	6	3 – Feature completion
6	Persistence: CreateMemento/SetMemento, schema/version validation	3	2	1	6	3 – Feature completion
7	Wire OnChunkSpawned + TL4 read-only outputs; integrate TL6 position read	3	2	1	6	4 – Integration
8	Unit, boundary, stress, integration tests + acceptance checklist	3	2	1	6	5 – System testing
9	Diagrams, documentation, and handoff package	2	1	1	4	6 – Final handoff

Total: 52 hours (26 planning, 17 build, 9 review across the 9 weeks).

Because this is a single contributor working a fixed number of hours per week, the nine work items run strictly in sequence – one week cannot start before the previous week ends. With uniform weekly durations, every week carries zero slack: all nine weeks are on the critical path. The PERT diagram and Gantt timeline below reflect this.

PERT:



All red: every week has zero slack. One contributor, fixed hours/week → strictly sequential, fully critical, 52 hrs total.

GANTT:

